Embedded System Lab Manual

Subject Name: Embedded System Lab

Subject Code: 7IT06

Experiment No.: 8

Demonstration of Inter-Process Communication (IPC) Simulation Using Arduino and Serial Monitor.

---

1. Objectives:

- To understand the concept of Inter-Process Communication (IPC).
- To simulate IPC using Arduino and the Serial Monitor.
- To demonstrate message exchange between two simulated processes using serial communication.

---

2. Outcomes:

After completing this experiment, students will be able to:

- Explain the concept and need for IPC in computing systems.
- Demonstrate basic communication between processes using serial communication.
- Simulate sender and receiver processes on an Arduino board.
- Analyze the communication flow using the Serial Monitor.

---

3. Requirements:

- Arduino UNO / Mega / Nano
- USB Cable
- Computer with Arduino IDE
- FreeRTOS Library for Arduino

---

4. Theory:

Inter-Process Communication (IPC) is a mechanism that allows processes to communicate with each other and synchronize their actions. It is crucial in systems

where multiple processes run concurrently. IPC mechanisms include pipes, message queues, semaphores, shared memory, etc.

In this simulation, we use serial communication between the Arduino and the Serial Monitor to mimic IPC. We create two pseudo-processes:

- ***Sender Process:*** Simulated by sending data (messages) through serial.
- ***Receiver Process:*** Simulated by reading and responding to the data.

This simulation is a simplified representation to help understand IPC without a full operating system-level implementation.

---

5. Procedure:

Step 1: Set up Arduino, and Open Arduino IDE.

Step 2: Write the simulation code to emulate two processes communicating.

Step 3: Upload the code to the Arduino board.

Step 4: Open Serial Monitor (ensure the correct baud rate is selected).

Step 5: Observe the communication between the simulated Sender and Receiver.

Step 6: Send input from the Serial Monitor (simulating sender process).

Step 7: Observe Arduino responding (receiver process logic).

Step 8: Record observations and interpret the communication.

---

6. Code:

```
/* IPC Simulation using Serial Communication on Arduino
Process 1: Sender (User via Serial Monitor)
Process 2: Receiver (Arduino reads and responds) */

void setup()
{
        Serial.begin(9600); // Initialize serial communication
        Serial.println("IPC Simulation Started");
        Serial.println("Type a message and press Enter:");
}
void loop()
{
```

```
    if (Serial.available() > 0)
    {
        String receivedData = Serial.readStringUntil('\n');        // Read input
        receivedData.trim();                                       // Remove unwanted spaces
        Serial.print("Sender (Process 1) sent: ");
        Serial.println(receivedData);

        // Simulating processing by Receiver (Process 2)
        String response = processData(receivedData);
        Serial.print("Receiver (Process 2) responded: ");
        Serial.println(response);

          Serial.println("\nType next message:");
    }
}

String processData(String input)
{
        // Simulated processing logic
        if (input.equalsIgnoreCase("hello"))
        {
            return "Hi there!";
        } else if (input.equalsIgnoreCase("time"))
        {
            return String("Current millis: ") + millis();
        } else if (input.equalsIgnoreCase("bye"))
        {
            return "Goodbye!";
        } else
        {
            return "Unknown command!";
        }
    }
```

7. Observations Table:

Sr. No.	Input from Sender (Serial Monitor)	Output from Receiver (Arduino)
1	hello	Hi there!
2	time	Current millis: 123456
3	abc	Unknown command!
4	bye	Goodbye!

8. Result:

The simulation successfully demonstrated basic Inter-Process Communication using Arduino and the Serial Monitor. The Arduino acted as a receiver process and responded appropriately to various commands entered by the user (sender process).

9. Conclusion:

This experiment illustrated the fundamental concept of IPC through a simple simulation on Arduino.